

Guía de Ejercicios Cross Validation

Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, mandenme un mail a rodriguezf@udesa.edu.ar y lo revisamos.

1. Ejercicios Conceptuales

1.1. Verdadero o Falso

1. Cross validation permite obtener una estimación más robusta del rendimiento del modelo.
2. En k-fold cross validation, el dataset se divide en k particiones exactamente iguales.
3. Leave-one-out cross validation es siempre la mejor opción para evaluar modelos.
4. Cross validation se puede usar tanto para seleccionar hiperparámetros como para evaluar el rendimiento final.
5. En 5-fold cross validation, cada observación se usa exactamente 4 veces para entrenar y 1 vez para validar.
6. Stratified cross validation es importante para datasets desbalanceados.
7. Cross validation siempre reduce el overfitting del modelo.
8. Un mayor número de folds siempre es mejor en cross validation.

1.2. Opción Múltiple

1. ¿Cuál es la principal ventaja de cross validation sobre hold-out validation?
 - a) Es más rápido computacionalmente
 - b) Usa toda la data para entrenamiento y validación

- c)* No requiere dividir los datos
 - d)* Es más fácil de implementar
- 2. En 10-fold cross validation, ¿qué porcentaje de los datos se usa para entrenar en cada fold?
 - a)* 10 %
 - b)* 80 %
 - c)* 90 %
 - d)* 100 %
- 3. ¿Cuándo es recomendable usar Leave-One-Out cross validation?
 - a)* Con datasets muy grandes
 - b)* Con datasets muy pequeños
 - c)* Siempre que sea posible
 - d)* Nunca
- 4. ¿Qué es stratified cross validation?
 - a)* Una técnica para datasets grandes
 - b)* Una forma de mantener la proporción de clases en cada fold
 - c)* Un método más rápido de validación
 - d)* Una técnica solo para regresión
- 5. ¿Cuál es el trade-off principal al elegir el número de folds?
 - a)* Sesgo vs Varianza
 - b)* Velocidad vs Precisión
 - c)* Complejidad vs Simplicidad
 - d)* Todas las anteriores

2. Ejercicios Prácticos

2.1. Ejercicio 1 - Cálculo de Modelos a Entrenar

Considerar los siguientes escenarios e indicar cuántos árboles es necesario entrenar en cada caso:

1. En el entrenamiento de un modelo CART se usa 5-fold cross validation para seleccionar el hiperparámetro `max_depth` de la siguiente grilla: `[none, 10, 20, 30]`
2. En el entrenamiento de un modelo CART se usa 5-fold cross validation para seleccionar la mejor combinación de los siguientes hiperparámetros:
 - `max_depth`: `[None, 10, 20, 30, 40, 50]`
 - `min_samples_split`: `[2, 5, 10]`
 - `min_samples_leaf`: `[1, 2, 4]`
3. En el entrenamiento de un modelo Random Forest se usa 5-fold cross validation para seleccionar la mejor combinación de los siguientes hiperparámetros:
 - `n_estimators`: `[100, 200, 500, 1000]`
 - `max_depth`: `[10, 20, 30]`
 - `min_samples_split`: `[2, 5, 10]`

2.2. Ejercicio 2 - Interpretación de Resultados

Tener en cuenta los siguientes resultados de la cross validation del hiperparámetro `n_estimators` para un modelo CART. ¿Qué valor del hiperparámetro elegirían? ¿Por qué?

```
1 {  
2 'mean_fit_time': array([7.5903286, 12.5109745, 30.5498374,  
   61.4766215]),  
3 'std_fit_time': array([2.8521561, 0.3776906, 0.5143247,  
   0.6323429]),  
4 'mean_score_time': array([0.1880916, 0.0871274, 0.9984307,  
   1.9107995]),
```

```

5 'std_score_time': array([0.0091651, 0.0154946, 0.1213947,
6   0.1533748]),
7 'param_n_estimators': masked_array(data=[100, 200, 500,
8   1000]),
9 'params': [{ 'n_estimators': 100},
10   { 'n_estimators': 200},
11   { 'n_estimators': 500},
12   { 'n_estimators': 1000}],
13 'split0_test_score': array([0.7995629, 0.7997190, 0.7998751,
14   0.7998751]),
15 'split1_test_score': array([0.7995629, 0.7997190, 0.7997190,
16   0.7997190]),
17 'split2_test_score': array([0.7995316, 0.7998438, 0.7998438,
18   0.7998438]),
19 'split3_test_score': array([0.7993754, 0.7996877, 0.7996877,
20   0.7998438]),
21 'split4_test_score': array([0.7996877, 0.7998438, 0.7998438,
22   0.7998438]),
23 'mean_test_score': array([0.7995754, 0.799762, 0.7997939,
24   0.7998251]),
25 'std_test_score': array([1.2270604e-04, 6.7250609e-05,
26   7.5455567e-05, 5.4430172e-05]),
27 'rank_test_score': array([4, 3, 2, 1], dtype=int32)
28 }

```

2.3. Ejercicio 3 - KNN y Cross Validation

Considerar el siguiente conjunto de datos donde se quiere predecir la etiqueta, + o - en función de la variable X. Con este conjunto de datos se quiere entrenar un modelo knn, para el cual el hiperparámetro k se seleccionará por cross-validation:

	1	2	3	4	5	6	7	8	9	10
X	-0.1	0.7	1.0	1.6	2.0	2.5	3.0	3.5	4.1	4.9
Y	-	+	+	-	+	+	-	-	+	+

1. Dar el accuracy de cross-validation por leave-one-out (es decir 10-folds cross-validation) de 1 vecino más cercano para la muestra de entrenamiento.
2. Dar el accuracy de cross-validation por leave-one-out (es decir 10-folds

cross-validation) de 3 vecinos más cercanos para la muestra de entrenamiento.

3. ¿Qué valor de k seleccionas?

2.4. Ejercicio 4 - Implementación de K-Fold

Implementá k-fold cross validation desde cero en Python para evaluar un modelo de regresión lineal:

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 from sklearn.metrics import mean_squared_error
4
5 def k_fold_cv(X, y, k=5, model_class=LinearRegression):
6     """
7     Implementa k-fold cross validation
8
9     Parameters:
10    X: features
11    y: target
12    k: numero de folds
13    model_class: clase del modelo a evaluar
14
15    Returns:
16    scores: lista de scores para cada fold
17    """
18    # Tu codigo aca
19    pass
20
21 # Datos de ejemplo
22 X = np.random.randn(100, 3)
23 y = X[:, 0] + 2*X[:, 1] - X[:, 2] + np.random.randn(100)*0.1
24
25 # Evaluar modelo
26 scores = k_fold_cv(X, y, k=5)
27 print(f"MSE promedio: {np.mean(scores):.4f}")
28 print(f"Desviacion estandar: {np.std(scores):.4f}")
```

2.5. Ejercicio 5 - Stratified Cross Validation

Tenemos un dataset desbalanceado con las siguientes clases:

Clase	Cantidad
A	800
B	150
C	50

1. ¿Por qué es importante usar stratified cross validation en este caso?
2. ¿Cuántas observaciones de cada clase habría en cada fold si usamos 5-fold stratified CV?
3. ¿Qué pasaría si usáramos k-fold simple sin estratificación?

2.6. Ejercicio 6 - Grid Search con Cross Validation

Implementá grid search con cross validation para optimizar hiperparámetros de un SVM:

```

1 from sklearn.svm import SVC
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.datasets import make_classification
4
5 # Generar datos
6 X, y = make_classification(n_samples=1000, n_features=10,
7                           n_classes=2, random_state=42)
8
9 # Definir grilla de parametros
10 param_grid = {
11     'C': [0.1, 1, 10, 100],
12     'gamma': ['scale', 'auto', 0.001, 0.01, 0.1],
13     'kernel': ['rbf', 'linear']
14 }
15
16 # Tu codigo aca para implementar grid search con CV

```

2.7. Ejercicio 7 - Comparación de Modelos

Usando cross validation, compará el rendimiento de los siguientes modelos en un problema de clasificación:

1. Logistic Regression

2. Random Forest

3. SVM

4. KNN

¿Qué métrica usarías? ¿Cómo determinarías si las diferencias son estadísticamente significativas?

2.8. Ejercicio 8 - Análisis de Varianza

1. ¿Por qué es importante reportar tanto la media como la desviación estándar de los scores de CV?
2. ¿Qué indica una alta varianza en los scores de cross validation?
3. ¿Cómo podríamos reducir la varianza en los resultados de CV?

3. Anexo: Soluciones

3.1. Soluciones - Verdadero o Falso

1. **Verdadero.** CV proporciona múltiples estimaciones del rendimiento, reduciendo la dependencia de una única división train/test.
2. **Falso.** Los folds pueden tener tamaños ligeramente diferentes si n no es divisible por k .
3. **Falso.** LOOCV puede tener alta varianza y ser computacionalmente costoso.
4. **Verdadero.** CV se usa tanto para model selection como para model evaluation.
5. **Verdadero.** En 5-fold CV, cada observación se usa 4 veces para entrenar y 1 vez para validar.
6. **Verdadero.** Mantiene la proporción de clases en cada fold.
7. **Falso.** CV ayuda a detectar overfitting, pero no lo reduce directamente.
8. **Falso.** Más folds aumenta la varianza y el costo computacional.

3.2. Soluciones - Opción Múltiple

1. **b)** Usa toda la data tanto para entrenamiento como para validación en diferentes momentos.
2. **c)** 90 % (9 de los 10 folds se usan para entrenamiento).
3. **b)** Con datasets muy pequeños, cuando queremos usar todos los datos posibles.
4. **b)** Mantiene la proporción de clases en cada fold.
5. **d)** Todas las anteriores: más folds aumentan sesgo pero reducen varianza, son más lentos pero más precisos, y más complejos pero potencialmente mejores.

3.3. Soluciones - Ejercicio 1

1. CART con 5-fold CV y 4 valores de max_depth:

- 4 valores de hiperparámetro $\times$ 5 folds = **20 árboles**

2. CART con 5-fold CV y múltiples hiperparámetros:

- max_depth: 6 valores
- min_samples_split: 3 valores
- min_samples_leaf: 3 valores
- Total combinaciones: $6 \times 3 \times 3 = 54$
- 54 combinaciones $\times$ 5 folds = **270 árboles**

3. Random Forest con 5-fold CV:

- n_estimators: 4 valores
- max_depth: 3 valores
- min_samples_split: 3 valores
- Total combinaciones: $4 \times 3 \times 3 = 36$
- 36 combinaciones $\times$ 5 folds = 180 Random Forests
- Cada Random Forest tiene su propio n_estimators (100, 200, 500, o 1000)
- Total de árboles individuales = $180 \times$ promedio de n_estimators
- Promedio de n_estimators = $(100+200+500+1000)/4 = 450$
- **Total aproximado: 81,000 árboles individuales**

3.4. Soluciones - Ejercicio 2

Analizando los resultados:

- **n_estimators = 100:** score = 0.7996, tiempo = 7.59s, std = 1.23e-04
- **n_estimators = 200:** score = 0.7998, tiempo = 12.51s, std = 6.73e-05
- **n_estimators = 500:** score = 0.7998, tiempo = 30.55s, std = 7.55e-05

- **n_estimators = 1000:** score = 0.7999, tiempo = 61.48s, std = 5.44e-05

Recomendación: n_estimators = 100

Justificación:

- La diferencia de score entre 100 y 200 es solo 0.0002 (0.7996 vs 0.7998)
- Esta diferencia es muy pequeña y probablemente no sea estadísticamente significativa
- El tiempo aumenta 65 % (de 7.59s a 12.51s) para una ganancia mínima
- La desviación estándar es ligeramente mayor en 100, pero aún muy baja
- En términos prácticos, 0.0002 de diferencia no justifica el costo computacional adicional

3.5. Soluciones - Ejercicio 3

Dataset ordenado por X:

Punto	1	2	3	4	5	6	7	8	9	10
X	-0.1	0.7	1.0	1.6	2.0	2.5	3.0	3.5	4.1	4.9
Y	-	+	+	-	+	+	-	-	+	+

a) LOOCV con k=1 (1-NN):

Para cada punto, encontramos su vecino más cercano y vemos si coincide la clase:

- Punto 1 (X=-0.1, Y=-): vecino más cercano es punto 2 (Y=+) → **Error**
- Punto 2 (X=0.7, Y=+): vecino más cercano es punto 3 (Y=+) → **Correcto**
- Punto 3 (X=1.0, Y=+): vecino más cercano es punto 2 (Y=+) → **Correcto**
- Punto 4 (X=1.6, Y=-): vecino más cercano es punto 5 (Y=+) → **Error**

- Punto 5 (X=2.0, Y=+): vecino más cercano es punto 4 (Y=-) → **Error**
- Punto 6 (X=2.5, Y=+): vecino más cercano es punto 5 (Y=+) → **Correcto**
- Punto 7 (X=3.0, Y=-): vecino más cercano es punto 6 (Y=+) → **Error**
- Punto 8 (X=3.5, Y=-): vecino más cercano es punto 7 (Y=-) → **Correcto**
- Punto 9 (X=4.1, Y=+): vecino más cercano es punto 8 (Y=-) → **Error**
- Punto 10 (X=4.9, Y=+): vecino más cercano es punto 9 (Y=+) → **Correcto**

Accuracy k=1: 6/10 = 0.60

b) LOOCV con k=3 (3-NN):

Para cada punto, encontramos sus 3 vecinos más cercanos y usamos votación por mayoría:

- Punto 1: vecinos [2,3,4] → clases [+,-,-] → mayoría - → **Error**
- Punto 2: vecinos [1,3,4] → clases [-,-,-] → mayoría - → **Error**
- Punto 3: vecinos [2,4,5] → clases [+,-,-] → mayoría - → **Error**
- Punto 4: vecinos [3,5,6] → clases [+,-,-] → mayoría - → **Error**
- Punto 5: vecinos [4,6,7] → clases [-,-,-] → mayoría - → **Error**
- Punto 6: vecinos [5,7,8] → clases [+,-,-] → mayoría - → **Error**
- Punto 7: vecinos [6,8,9] → clases [+,-,-] → mayoría - → **Error**
- Punto 8: vecinos [7,9,10] → clases [-,-,-] → mayoría - → **Error**
- Punto 9: vecinos [8,10,7] → clases [-,-,-] → mayoría - → **Error**
- Punto 10: vecinos [9,8,7] → clases [-,-,-] → mayoría - → **Error**

Accuracy k=3: 1/10 = 0.10

c) Selección de k: Elegiría **k=1** porque tiene mejor accuracy (0.60 vs 0.10). Con este dataset pequeño y ruidoso, k=3 incluye demasiados vecinos que añaden ruido a la predicción.

3.6. Soluciones - Ejercicio 4

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 from sklearn.metrics import mean_squared_error
4
5 def k_fold_cv(X, y, k=5, model_class=LinearRegression):
6     """
7     Implementa k-fold cross validation
8     """
9     n_samples = len(X)
10    fold_size = n_samples // k
11    indices = np.arange(n_samples)
12    np.random.shuffle(indices) # mezclar indices
13
14    scores = []
15
16    for i in range(k):
17        # Definir indices de validacion
18        start_idx = i * fold_size
19        end_idx = start_idx + fold_size if i < k-1 else
n_samples
20        val_indices = indices[start_idx:end_idx]
21        train_indices = np.concatenate([indices[:start_idx],
22                                       indices[end_idx:]])
23
24        # Dividir datos
25        X_train, X_val = X[train_indices], X[val_indices]
26        y_train, y_val = y[train_indices], y[val_indices]
27
28        # Entrenar modelo
29        model = model_class()
30        model.fit(X_train, y_train)
31
32        # Predecir y evaluar
33        y_pred = model.predict(X_val)
34        score = mean_squared_error(y_val, y_pred)
35        scores.append(score)
36
37    return scores
```

3.7. Soluciones - Ejercicio 5

1. **Importancia de stratified CV:** Con clases desbalanceadas, k-fold simple podría crear folds sin representación de clases minoritarias. Stratified CV garantiza que cada fold mantenga la proporción original de clases.
2. **Observaciones por fold en 5-fold stratified CV:**
 - Total: 1000 observaciones
 - Proporción: A=80 %, B=15 %, C=5 %
 - Por fold (200 obs): A=160, B=30, C=10
3. **Problema sin estratificación:** Algunos folds podrían tener muy pocas o ninguna observación de clases B y C, dando estimaciones sesgadas del rendimiento.

3.8. Soluciones - Ejercicio 6

```
1 from sklearn.svm import SVC
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.datasets import make_classification
4
5 # Generar datos
6 X, y = make_classification(n_samples=1000, n_features=10,
7                             n_classes=2, random_state=42)
8
9 # Definir grilla de parametros
10 param_grid = {
11     'C': [0.1, 1, 10, 100],
12     'gamma': ['scale', 'auto', 0.001, 0.01, 0.1],
13     'kernel': ['rbf', 'linear']
14 }
15
16 # Grid search con CV
17 grid_search = GridSearchCV(
18     SVC(),
19     param_grid,
20     cv=5,
21     scoring='accuracy',
22     n_jobs=-1,
23     verbose=1
```

```

24 )
25
26 grid_search.fit(X, y)
27
28 print(f"Mejores parametros: {grid_search.best_params_}")
29 print(f"Mejor score: {grid_search.best_score_:.4f}")

```

3.9. Soluciones - Ejercicio 7

```

1 from sklearn.model_selection import cross_val_score
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.svm import SVC
5 from sklearn.neighbors import KNeighborsClassifier
6 import numpy as np
7
8 # Modelos a comparar
9 models = {
10     'Logistic Regression': LogisticRegression(),
11     'Random Forest': RandomForestClassifier(),
12     'SVM': SVC(),
13     'KNN': KNeighborsClassifier()
14 }
15
16 # Comparar modelos
17 results = {}
18 for name, model in models.items():
19     scores = cross_val_score(model, X, y, cv=5, scoring='
f1_weighted')
20     results[name] = scores
21     print(f"{name}: {scores.mean():.4f} (+/- {scores.std()
*2:.4f})")
22
23 # Test estadístico (t-test pareado) para comparar modelos
24 from scipy import stats
25 model1_scores = results['Random Forest']
26 model2_scores = results['SVM']
27 t_stat, p_value = stats.ttest_rel(model1_scores,
    model2_scores)
28 print(f"p-value: {p_value:.4f}")

```

Usaría **F1-score weighted** para datasets desbalanceados o **accuracy** para balanceados. Para significancia estadística, usaría t-test pareado entre scores de CV.

3.10. Soluciones - Ejercicio 8

1. **Importancia de media y desviación:** La media indica el rendimiento esperado, la desviación indica la estabilidad del modelo. Ambas son cruciales para tomar decisiones informadas.
2. **Alta varianza indica:**
 - Modelo sensible a cambios en datos de entrenamiento
 - Posible overfitting
 - Dataset pequeño o muy heterogéneo
 - Necesidad de regularización
3. **Reducir varianza:**
 - Aumentar el dataset
 - Usar modelos más simples
 - Aplicar regularización
 - Aumentar el número de folds (trade-off con costo computacional)
 - Usar stratified CV para datasets desbalanceados